

SQL Window Functions

14 · Window Functions Basics — Complete Reference Guide

Table of Contents

1. Window Basics
2. OVER Clause
3. PARTITION Clause
4. ORDER Clause
5. FRAME Clause
6. Window Rules
7. Window with GROUP BY

1. Window Functions — Basics

→ *Task 1: Total sales across all orders (standard aggregate)*

```
SELECT SUM(Sales) AS Total_Sales
FROM Sales.Orders;
```

→ *Task 2: Total sales for each product (standard GROUP BY)*

```
SELECT
    ProductID,
    SUM(Sales) AS Total_Sales
FROM Sales.Orders
GROUP BY ProductID;
```

2. OVER Clause — Window Aggregate

→ *Task 3: Total sales across all orders — keep row-level detail*

```
SELECT
    OrderID,
    OrderDate,
    ProductID,
```

```
Sales,  
SUM(Sales) OVER () AS Total_Sales  
FROM Sales.Orders;
```

3. PARTITION Clause

→ *Task 4: Total sales overall AND per product*

```
SELECT  
    OrderID, OrderDate, ProductID, Sales,  
    SUM(Sales) OVER () AS Total_Sales,  
    SUM(Sales) OVER (PARTITION BY ProductID) AS Sales_By_Product  
FROM Sales.Orders;
```

→ *Task 5: Total sales overall, per product, and per product+status*

```
SELECT  
    OrderID, OrderDate, ProductID, OrderStatus, Sales,  
    SUM(Sales) OVER () AS Total_Sales,  
    SUM(Sales) OVER (PARTITION BY ProductID) AS Sales_By_Product,  
    SUM(Sales) OVER (PARTITION BY ProductID, OrderStatus) AS Sales_By_Product_Status  
FROM Sales.Orders;
```

4. ORDER Clause — Ranking

→ *Task 6: Rank each order by Sales from highest to lowest*

```
SELECT  
    OrderID, OrderDate, Sales,  
    RANK() OVER (ORDER BY Sales DESC) AS Rank_Sales  
FROM Sales.Orders;
```

5. FRAME Clause

→ *Task 7: Running total — current row and next 2 rows*

```
SELECT  
    OrderID, OrderDate, OrderStatus, Sales,  
    SUM(Sales) OVER (  
        PARTITION BY OrderStatus  
        ORDER BY OrderDate  
        ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING  
    ) AS Total_Sales  
FROM Sales.Orders;
```

→ *Task 8: Rolling total — previous 2 rows and current row*

```
SELECT  
    OrderID, OrderDate, OrderStatus, Sales,  
    SUM(Sales) OVER (  
        PARTITION BY OrderStatus  
        ORDER BY OrderDate  
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
    ) AS Total_Sales  
FROM Sales.Orders;
```

```

        PARTITION BY OrderStatus
        ORDER BY OrderDate
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS Total_Sales
FROM Sales.Orders;

```

→ *Task 9: Previous 2 rows only*

```

SELECT
    OrderID, OrderDate, OrderStatus, Sales,
    SUM(Sales) OVER (
        PARTITION BY OrderStatus
        ORDER BY OrderDate
        ROWS 2 PRECEDING
    ) AS Total_Sales
FROM Sales.Orders;

```

→ *Task 10 & 11: Cumulative total from start to current row*

```

SELECT
    OrderID, OrderDate, OrderStatus, Sales,
    SUM(Sales) OVER (
        PARTITION BY OrderStatus
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Total_Sales
FROM Sales.Orders;

```

6. Window Function Rules

Rule 1 — Window functions can ONLY be used in SELECT or ORDER BY clauses.

They CANNOT be used in WHERE, HAVING, or GROUP BY.

```

-- INVALID: Window function in WHERE clause
SELECT OrderID, Sales,
    SUM(Sales) OVER (PARTITION BY OrderStatus) AS Total_Sales
FROM Sales.Orders
WHERE SUM(Sales) OVER (PARTITION BY OrderStatus) > 100; -- ERROR

```

Rule 2 — Window functions CANNOT be nested inside other window functions.

```

-- INVALID: Nested window functions
SELECT
    SUM(SUM(Sales) OVER (PARTITION BY OrderStatus))
    OVER (PARTITION BY OrderStatus) AS Total_Sales -- ERROR
FROM Sales.Orders;

```

7. Window Functions with GROUP BY

→ *Task 12: Rank customers by their total sales*

```
SELECT
    CustomerID,
    SUM(Sales) AS Total_Sales,
    RANK() OVER (ORDER BY SUM(Sales) DESC) AS Rank_Customers
FROM Sales.Orders
GROUP BY CustomerID;
```